UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	M. Rezki Riopaldy
Kelas	SI5C
Judul project / topik API	RESTful API Kepegawaian
Tautan repositori	https://github.com/rezkiriopaldy-secret-V/tugas1-restful-2428240117.git
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	14 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Sebelum mengambil screenshot Runner
Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:

RESTful API Kepegawaian merupakan API yang digunakan untuk mengelola data karyawan, meliputi NIP, nama, jabatan, departemen, dan gaji. API dikembangkan menggunakan Express.js dan menyediakan operasi CRUD (Create, Read, Update, Delete) untuk menampilkan, menambahkan, memperbarui, dan menghapus data karyawan. Pengujian endpoint dilakukan menggunakan Postman dengan automated test script.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/employees	Tidak	200	Menampilkan seluruh data karyawan
2	GET	/employees/:id	Tidak	200	Menampilkan data karyawan berdasarkan ID
3	POST	/employees	Tidak	201	Menambahkan data karyawan
4	PUT	/employees/:id	Tidak	200	Memperbarui data karyawan
5	DELETE	/employees/:id	tidak	200	Menghapus data karyawan
6					
7					
8					

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

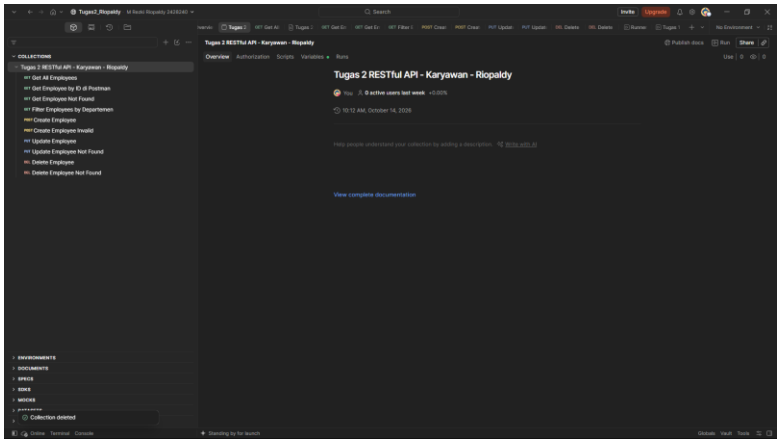
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat server Lokal
2	employeeId	Diisi otomatis oleh script POST, jika sudah diterapkan	Menyimpan ID karyawan baru untuk request DELETE

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:

Request dalam Collection Postman dikelompokkan berdasarkan fungsi CRUD dan skenario pengujian. Pengelompokan dilakukan dengan mengurutkan request berdasarkan metode HTTP, yaitu GET untuk mengambil data, POST untuk menambahkan data, PUT untuk memperbarui data, dan DELETE untuk menghapus data. Setiap metode dilengkapi dengan skenario pengujian gagal, seperti data tidak ditemukan dan input tidak valid.

Urutan request dimulai dari pengambilan seluruh data karyawan, pengambilan data berdasarkan ID, pengujian data tidak ditemukan, dan filter berdasarkan departemen. Selanjutnya, dilakukan pengujian penambahan data, input tidak valid, pembaruan data, pembaruan data yang tidak ditemukan, penghapusan data, dan penghapusan data yang tidak ditemukan. Urutan ini dipilih agar pengujian berjalan secara sistematis, mulai dari pengujian dasar hingga pengujian kondisi kesalahan. Selain itu, pengujian penambahan data dilakukan sebelum penghapusan data agar ID karyawan yang baru dibuat dapat digunakan dalam proses penghapusan.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	GET all Employees	GET	{{baseUrl}}/employees	Utama
2	GET Employees by ID	GET	{{baseUrl}}/employees/1	Utama
3	GET Employess Not Found	GET	{{baseUrl}}/employees/99	Skenario Gagal

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
4	Filter Employees by Departemen	GET	{{baseUrl}}/employees?departemen=Keuangan	Utama
5	Create Employess	POST	{{baseUrl}}/employees	CRUD
6	Create Employee Invalid	POST	{{baseUrl}}/employees	Skenario Gagal
7	Update Employee	PUT	{{baseUrl}}/employees/1	CRUD
8	Update Employee Not Found	PUT	{{baseUrl}}/employees/99	Skenario Gagal
9	Delete Employee	DELETE	{{baseUrl}}/employees/{{employeeId}} jika variabel sudah diterapkan	CRUD
10	Delete Employee Not Found	DELETE	{{baseUrl}}/employees/99	Skenario Gagal

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	Get all Employees	Status 200	
2	Get Employees by ID	Status 200	
3	Get Employees Not Found	Status 404	
4	Filter Employees by Departemen	Status 200	
5	Create Employees	Status 201	
6	Create Employees Invalid	Status 400	
7	Update Employees	Status 200	
8	Update Employees Not Found	Status 404	
9	Delete Employees	Status 200	
10	Delete Employees Not Found	Status 404	

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Response berisi array karyawan", function () {
    const data = pm.response.json();
    pm.expect(data).to.be.an("array");
});

pm.test("Data karyawan tidak kosong", function () {
    const data = pm.response.json();
    pm.expect(data.length).to.be.greaterThan(0);
});
```

Penjelasan:

Script pada request Get All Employees memeriksa bahwa API memberikan status HTTP 200, mengembalikan data dalam bentuk array, dan memiliki minimal satu data karyawan. Pengujian ini memastikan endpoint dapat diakses dan menghasilkan struktur respons yang sesuai dengan kebutuhan.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
pm.test("Status code is 201", function () {
    pm.response.to.have.status(201);
});

const responseData = pm.response.json();

pm.test("Data karyawan berhasil ditambahkan", function () {
    pm.expect(responseData.status).to.eql("success");
    pm.expect(responseData.data.nama).to.eql("Andi Saputra");
    pm.expect(responseData.data.nip).to.eql("EMP-0048");
    pm.expect(responseData.data.id).to.exist;
});

if (pm.response.code === 201 && responseData.data && responseData.data.id) {
    pm.collectionVariables.set("employeeId", responseData.data.id);
}
```

Penjelasan:

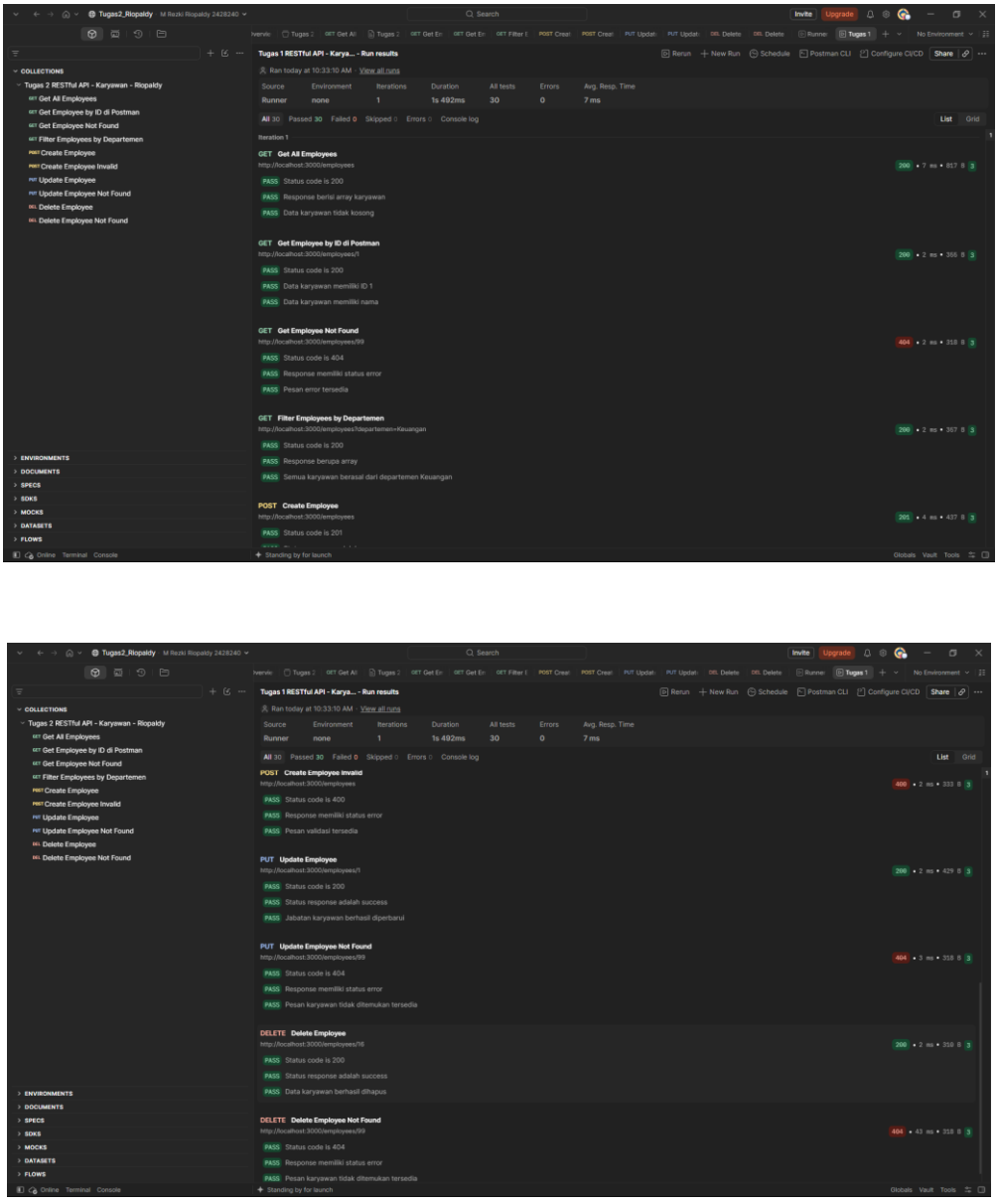
Pada request Create Employee, script memeriksa status HTTP 201 dan memastikan data karyawan berhasil ditambahkan. ID karyawan yang diperoleh dari respons disimpan ke variabel Collection bernama employeeId. Variabel tersebut digunakan oleh request Delete Employee agar penghapusan mengacu pada ID karyawan yang benar. Oleh karena itu, Create Employee harus dijalankan sebelum Delete Employee.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:

Pengujian RESTful API Kepegawaian dilakukan menggunakan fitur Collection Runner pada Postman dengan menjalankan 10 request yang telah disusun dalam Collection. Request tersebut mencakup pengujian pengambilan data, penambahan data, pembaruan data, penghapusan data, serta skenario kegagalan seperti data tidak ditemukan dan input tidak valid.

Sebelum menjalankan Collection Runner, server API lokal dinyalakan kembali untuk memastikan aplikasi dapat menerima request. Selanjutnya, dipastikan bahwa URL dasar API telah dikonfigurasi dengan benar dan seluruh request beserta skrip pengujiannya telah disiapkan. Pengujian kemudian dijalankan secara berurutan agar proses yang bergantung pada data sebelumnya, seperti penambahan dan penghapusan data karyawan, dapat berjalan dengan baik.

Hasil pengujian dicatat berdasarkan jumlah request yang dijalankan, jumlah test yang lulus (passed), dan jumlah test yang gagal (failed) sebagaimana ditampilkan pada hasil Collection Runner. Pengujian dinyatakan berhasil apabila seluruh test lulus dan tidak terdapat test yang gagal.

4.2 Rekap angka hasil

Nama Collection	Tugas 2 RESTful API - Karyawan - Riopaldy
Environment yang dipakai	Not environment
Jumlah request dijalankan	10
Jumlah test lulus	30
Jumlah test gagal	0
Tanggal dan jam pengujian	14 Oktober 2026 - 11:06

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	Delete Employee	Status yang diharapkan 200, tetapi API mengembalikan 404	ID karyawan yang hendak dihapus tidak ditemukan saat Collection Runner dijalankan	Menyimpan ID dari respons Create Employee ke variabel `employeeId` dan menggunakannya pada URL Delete Employee

Bab 5 Repositori dan Berkas

Tautan repositori	https://github.com/rezkiriopaldy-secret-V/tugas1-restful-2428240117.git
Lokasi berkas Collection (.json)	Tugas 2 RESTful API - Karyawan - Riopaldy.postman_collection.json
Repositori dapat dibuka dosen	[Y] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:

Assertion yang paling berguna adalah pemeriksaan status code dan isi respons JSON, karena keduanya membantu memastikan API memberikan hasil yang sesuai dengan harapan. Pada tugas ini, assertion status code membantu menemukan ketidaksesuaian respons, sedangkan pemeriksaan isi JSON memastikan pesan dan data karyawan yang dikembalikan sudah benar.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:

Pengujian manual mengharuskan saya menjalankan request satu per satu, sedangkan Collection Runner dapat menjalankan seluruh request secara otomatis dan menampilkan hasil pengujiannya. Bagian tersulit adalah menulis test script dan memastikan ID karyawan digunakan dengan benar pada request berikutnya, sehingga saya mengatasinya dengan memeriksa respons JSON dan menyimpan ID dari request Create Employee ke variabel employeeId untuk digunakan pada request Delete Employee.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	Ya
2	Setiap endpoint API saya ada di Collection	Ya
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	Ya
4	Setiap request punya assertion status code dan struktur JSON	Ya
5	Nama setiap test berupa kalimat yang menyatakan harapan	Ya
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	Ya
7	Server dinyalakan ulang sebelum Runner dijalankan	Ya
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	Ya
9	Berkas Environment tidak memuat kunci rahasia sungguhan	Ya
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	Ya

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen